Nichelle Graf
L04: Chihuahua or Muffin Workshop Reflection
ITA1378 Computer Vision AI
Professor Patricia McManus

Lab 04 - Chihuahua or Muffin Workshop Reflective Journal

Summary of the Workshop

The objective of this workshop was to build and evaluate a basic neural network capable of classifying images as either a Chihuahua or a muffin. Although the task sounds humorous, it demonstrates a real computer vision challenge: objects from completely different categories can share similar visual characteristics such as color, shape, and texture. The workshop provided hands-on experience with the complete process of preparing image data, defining a neural network, training the model, and evaluating its predictions on previously unseen images.
The first step was cloning the provided GitHub repository into Google Colab so that the notebook could access the image dataset and supporting files. The data was divided into training and validation folders, with separate Chihuahua and muffin subfolders. The workshop contained 120 training images and 30 validation images, representing an 80/20 split.

Before the images could be used by the model, they needed to be standardized. The images originally had different dimensions, so they were resized to 224 × 224 pixels. They were then converted to PyTorch tensors and normalized. This demonstrated why preprocessing is an important part of computer vision: a neural network needs consistent numerical input even when the original images vary considerably in size and appearance.

Key Concepts Learned

One of the most important concepts in this workshop was how image data is represented as input to a neural network. Each RGB image contained three color channels and was resized to a shape of 3 × 224 × 224. The network used in this lab was a basic fully connected neural network rather than a convolutional neural network. The image tensor was flattened into one long vector before being passed through layers containing 128, 64, and 32 neurons, followed by an output layer with two possible classes.

The network was trained using cross-entropy loss and stochastic gradient descent. The loss function measured how far the model's predictions were from the correct labels, while the optimizer used that feedback to update the model's weights. During training, the model performed a forward pass to generate predictions and then used backpropagation to calculate how the network weights should be adjusted.

Another important concept was the separation between training and validation data. Training performance indicates how well the model learns the examples it has already seen, while validation performance provides a better indication of how well it can generalize to new images.

The journal instructions mention concepts such as convolutional neural networks and transfer learning as possible examples, but this particular workshop focused on a basic neural network. That distinction was useful because it provides a baseline that can later be compared with a CNN-based approach.

Challenges Encountered

The main challenge I experienced was recognizing which parts of the notebook were intentionally incomplete. Several code cells contained question marks as placeholders, which initially produced syntax errors when executed. For example, the image height and width had to be determined from the network diagram and replaced with 224 and 224. Other placeholders required inserting the correct training and validation transformations and specifying the batch size.

Once I understood that these were intended fill-in-the-blank exercises rather than errors in the notebook, the process became more straightforward. I also encountered warning messages during training related to the progress-bar library and the number of DataLoader workers. These did not prevent the model from training successfully, so I learned the importance of distinguishing between warnings and actual execution errors.

I also noticed a small error in the notebook's display code where the validation dataset print statement referenced the training dataset. It did not affect training, but identifying it reinforced the importance of reading code carefully rather than assuming all provided code is correct.

Insights from Model Performance

The most interesting part of the workshop was examining the difference between training accuracy and validation accuracy. After three epochs, training accuracy increased to approximately 90 percent. However, validation accuracy peaked earlier, at approximately 83 percent during the second epoch, and then dropped to about 73 percent during the third epoch.

This suggests that the model was beginning to overfit. It was becoming increasingly successful at recognizing the training images while becoming less successful on images it had not previously seen. This reinforced a concept from earlier machine learning work: higher training accuracy does not necessarily mean a better model.

The prediction visualizations provided an even clearer example. Several Chihuahua images were incorrectly classified as muffins, sometimes with confidence levels above 95 or even 99 percent. The model generally appeared to perform better on muffins than on Chihuahuas. Some tightly cropped Chihuahua faces shared colors, rounded shapes, and visual textures with muffins, which likely contributed to the confusion.

One of my strongest takeaways from this exercise is that model confidence does not equal model correctness. A prediction can be extremely confident and still be wrong. Overall accuracy can also hide differences in performance between individual classes. Metrics such as a confusion matrix, precision, recall, and class-specific accuracy would provide a more complete evaluation.

Real-World Applications

The same basic image-classification process used in this workshop has applications far beyond distinguishing dogs from baked goods. Image classifiers are used in manufacturing quality control, medical imaging, automated recycling systems, agriculture, retail, security, and many other fields. In each case, a model must learn visual patterns and determine which category an image belongs to.

The Chihuahua-versus-muffin example also illustrates a serious real-world concern. If a model can make highly confident mistakes on visually similar images, confidence scores alone should not be treated as proof that an AI system is reliable. In higher-risk applications such as medical diagnosis or autonomous systems, incorrect but confident predictions could have significant consequences.

Personal Reflection

This workshop helped make the limitations of a simple neural network much more concrete. Before seeing the predictions, it would have been easy to focus only on the training accuracy and assume that 90 percent represented strong performance. Looking at the validation results and individual images told a much more complicated story.

I was especially interested in seeing how quickly the model began to overfit despite the simplicity of the task. The small dataset likely contributed to this problem, but the network architecture also has limitations. Flattening the entire image into a single vector removes much of the spatial relationship between neighboring pixels. The network receives all of the pixel values, but it does not specifically preserve concepts such as edges, shapes, or local patterns.

References

PyTorch. *Torchvision Transforms Documentation*. PyTorch Documentation.

PyTorch. *PyTorch Documentation*. PyTorch.

Timoner, P. *Workshop: Chihuahua vs. Muffin*. GitHub repository, patitimoner/workshop-chihuahua-vs-muffin.

AI Use Disclosure: ChatGPT was used as a learning aid to help organize my observations from the completed workshop and clarify technical concepts. The analysis and reflections are based on my own lab results and learning experience.